



Product Requirements Document

Adaptive rebalancing infrastructure for multi-asset onchain vaults

STATUS
PRD v1.1 - Core product frozen. Accounting boundary clarified after Concrete follow-up: Setpoint assumes vault accounting is solved upstream and focuses only on policy, solving, and rebalance orchestration. Three implementation variables remain open: first ICP subtype, exact solver objective, and first execution adapter.

Field	Value
Product	Setpoint
Document version	1.1
Date	26 September 2026
Owner	Emin Karagöz
Build context	Arbitrum Open House Singapore Buildathon / Founder House application
Initial ecosystem	Robinhood Chain + Arbitrum
Initial vertical	RWA / tokenized-stock index and multi-asset vaults
First external integration target	RWA Index on Robinhood Chain testnet
Repository	github.com/karagozemin/setpoint
Document classification	Internal working document; review before public sharing

Portfolio policy in. Safe executable rebalance out.

Contents

- 1. Executive summary
- 2. Product thesis and decision freeze
- 3. Origin, pivots, and why Setpoint exists
- 4. Problem definition
- 5. Users, ICP, and jobs to be done
- 6. Validation evidence and research synthesis
- 7. Product principles
- 8. Scope: MVP, V1, future, and explicit non-goals
- 9. Product experience and user flows
- 10. Functional requirements
- 11. Solver specification
- 12. Vault model, accounting, and priceability
- 13. Execution architecture and adapters
- 14. Security and threat model
- 15. Technical architecture
- 16. Data model and interfaces
- 17. RWA Index integration plan
- 18. Benchmark and evaluation framework
- 19. Testing and quality requirements
- 20. Observability, analytics, and success metrics
- 21. GTM, design partners, and business model hypotheses
- 22. Competitive positioning
- 23. Build plan and milestone schedule
- 24. Demo, judging, and Founder House narrative
- 25. Risks, kill criteria, and open decisions
- 26. Definition of done
- Appendix A. RWA Index technical facts
- Appendix B. Validation log
- Appendix C. Decision log
- Appendix D. Submission checklist

How to use this PRD

Sections 1-18 define the build. Sections 19-26 define how it is proven, shipped, and presented. Any proposed feature that conflicts with the MVP non-goals or weakens the core benchmark should be deferred.

1. Executive summary

Setpoint is an adaptive rebalance solver and orchestration layer for multi-asset onchain vaults. A curator defines portfolio targets or target ranges, risk constraints, and permitted assets. Setpoint reads the current vault state, pricing inputs, and available execution liquidity, then computes the best next executable rebalance step. After execution, Setpoint re-reads the resulting portfolio and recomputes rather than assuming an entire rebalance can be planned once and executed unchanged.

The initial product is focused on RWA and tokenized-stock vaults on Robinhood Chain, where portfolio rebalancing must deal with fragmented liquidity, market-hour asymmetry, oracle constraints, and the need to keep vault shares priceable throughout a transition. Setpoint assumes custody, NAV/share accounting, and deposit/redemption accounting are already solved by the integrated vault. It consumes those authoritative outputs; it does not invent a new accounting model. Existing execution infrastructure such as CoW Protocol, 0x, or vault-native adapters is used underneath Setpoint whenever practical.

Core promise

Given current portfolio state + target ranges + risk limits + prices + executable liquidity, Setpoint computes the safest and most efficient next rebalance batch, keeps the portfolio in an explicit priceable state, and re-evaluates after every completed execution.

1.1 Initial proof

The first external test case is an existing Robinhood Chain testnet RWA Index vault whose founder shared the repository, deployed addresses, rebalance interface, guard conditions, and fork-testing instructions. The first measurable proof is not a marketing demo: it is a controlled benchmark of the vault's current static per-leg sizing approach against Setpoint's adaptive sizing and orchestration.

1.2 What must be true for Setpoint to deserve to exist

- Vault managers repeatedly face execution inefficiency or operational risk that is not solved by existing swap routing alone.
- Adaptive sizing and stepwise re-optimization must produce a measurable benefit against a simple static baseline.
- The integrated vault must expose authoritative accounting and a priceable state after every completed step; Setpoint must not create, reconcile, or replace the vault's own NAV/share accounting.
- Setpoint must add decision intelligence above CoW/0x rather than duplicate their routing and settlement primitives.
- The initial integration must work against a real external vault interface, not a custom demo contract built only for Setpoint.

2. Product thesis and decision freeze

2.1 Frozen thesis

Frozen product definition

Setpoint is a non-custodial, policy-driven adaptive rebalance solver/orchestrator for multi-asset vaults. It converts portfolio targets and constraints into the best next executable rebalance step using current state, prices, and liquidity, then re-solves from the resulting state.

2.2 Frozen product decisions

Decision	Frozen position
Product layer	Portfolio policy + solver + orchestration. Not a DEX and not a custody product.
Primary problem	Static or brittle rebalance execution that ignores live depth, changing prices, and intermediate-state constraints.
Initial vertical	RWA / tokenized-stock and multi-asset vaults.
Initial chain	Robinhood Chain testnet for first integration; architecture remains Arbitrum-ecosystem compatible.
Portfolio target model	Target ranges / acceptable regions are preferred over exact final weights.
Execution model	Stepwise. Each completed step leaves an explicit, priceable portfolio state. Recompute before the next step.
Sizing	Liquidity/depth aware, not fixed percentage-per-leg.
External execution	Use existing trusted execution primitives or adapters where possible.
Accounting boundary	Solved upstream. Setpoint reads the vault's authoritative NAV/base-asset/share-accounting model. MVP supports a single numeraire; independent sleeves/sub-vaults are compatible when each sleeve has explicit accounting.
Cross-vault netting	Not core. Possible future optimization only.
Corporate actions	Safety module, not core product.
Market hours	Important tokenized-stock module; not required to prove solver v1.
Custody	Setpoint should not take custody of user funds in the core architecture.

2.3 Intentionally open implementation variables

- First ICP subtype: public ERC-4626 style vault versus managed/partner vault. Build must support the public-vault test case while research continues.
- Exact solver objective: the MVP prioritizes feasibility and drift reduction; the final weighting of execution cost, price impact, time-to-target, and turnover remains tunable.
- First production-grade external execution adapter: CoW, 0x, or a native vault adapter. The RWA Index test harness can precede this decision.

2.4 Re-opening the core thesis

The product thesis should only be re-opened if strong evidence falsifies it. Examples: multiple independent vault operators state that the problem is already solved and immaterial; the benchmark shows no meaningful

improvement against the baseline across credible liquidity scenarios; or an existing protocol is found to already provide the same policy, adaptive solving, re-evaluation, and vault integration workflow at production quality.

3. Origin, pivots, and why Setpoint exists

Setpoint did not begin as a rebalance solver. The initial concept was a portfolio-intent clearing network: multiple vaults would submit desired final portfolio states, opposing flows would be internally crossed, and only residual exposure would be sent to external liquidity. User research progressively removed assumptions that did not need to belong in the core product.

3.1 Pivot 1: opposing flow cannot be the product dependency

Feedback from a Gauntlet product perspective highlighted that vaults rebalance at widely different cadences, response time itself can be an edge, and rational curators may often trade in the same direction. A product that waits for counterflow risks delaying execution exactly when speed matters. Cross-portfolio netting may still save cost at scale, but it cannot be the condition under which the product creates value.

3.2 Pivot 2: the recurring pain is execution sizing under constraints

A Robinhood Chain RWA index-vault founder described a concrete failure mode: a fixed 10% NAV per-leg cap was too conservative for deep pools and too aggressive for thin pools. Thin liquidity caused simulations to fail against oracle/slippage guards; deep liquidity forced unnecessary extra cycles. Their stated preference was a solver that accepts current weights, targets, constraints, and available liquidity and returns the biggest safe batch that converges toward target.

3.3 Pivot 3: final state is not enough; every completed state must be priceable

Further feedback emphasized that vault positions must remain priceable during a rebalance. An opaque asynchronous promise to reach a final state later is insufficient for a public vault whose NAV, shares, deposits, and redemptions may need valuation throughout the process. Setpoint therefore optimizes one executable transition at a time, leaving the vault in a concrete state after every completed batch.

3.4 Pivot 4: automate policy, do not reinvent settlement

CoW feedback suggested that automatic self-balancing portfolio logic is meaningfully different from ordinary swaps or TWAPs, while existing primitives can and should handle vulnerable fund-moving operations. 0x support independently described the architectural boundary similarly: the application determines what needs to be traded, while 0x handles routing and execution for individual swaps. Setpoint is therefore the intelligence and orchestration layer above execution venues.

3.5 Pivot 5: vault accounting is upstream, not Setpoint scope

Concrete follow-up clarified that generic multi-asset accounting can become a product of its own. A vault that accepts economically different assets may be simpler as independent sleeves/sub-vaults, while a stable-vault experience can normalize deposits through an upstream swap-and-deposit path into one strategy asset. Setpoint therefore does not solve share accounting, reward accounting, reconciliation, or arbitrary deposit conversion. It integrates only where the vault already exposes a clean, authoritative valuation model and then optimizes the rebalance decisions above that boundary.

Result of the research loop

The product moved from “find opposing vault flow” to “continuously compute the best safe rebalance step for one vault even when no other vault exists,” while explicitly leaving custody and vault accounting upstream. That is the core Setpoint wedge.

4. Problem definition

4.1 Primary problem statement

Multi-asset vault curators know the portfolio they want to hold, but execution is not equivalent to subtracting current weights from target weights. For vaults that already expose reliable NAV/share accounting, real rebalances are constrained by pool depth, price impact, oracle rules, slippage limits, vault guardrails, market availability, cash buffers, and the requirement that the vault remain correctly valued throughout the transition. Static per-leg limits or one-shot plans can be too conservative in liquid markets and unsafe or unexecutable in thin ones.

4.2 User pain

- Sizing: a fixed percentage or fixed notional does not reflect current executable depth.
- Routing: the best venue and trade mode can differ by asset, size, and time.
- Convergence: conservative caps create unnecessary cycles and carry off-target exposure longer than necessary.
- Failure recovery: quotes expire, liquidity moves, and one completed leg invalidates assumptions for the next.
- Priceability: public-vault NAV and share accounting cannot depend on an opaque future final state.
- Policy safety: target changes themselves are a security surface; a malicious target can route a vault into an attacker-controlled asset without "stealing" through a traditional exploit.
- Tokenized-stock specifics: underlying market hours, stale/last-close references, and corporate actions can make an otherwise valid execution policy unsafe or stuck.

4.3 Why existing swap routers are insufficient by themselves

Routers such as 0x optimize an individual swap. CoW provides sophisticated order and settlement primitives. Neither boundary, by itself, is the same as continuously deciding how a portfolio should move from its current state toward a constrained target. Setpoint owns portfolio-level state, policy, and next-step decisions; execution venues own liquidity access and trade settlement.

5. Users, ICP, and jobs to be done

5.1 Primary user

The primary user is the person or automated agent responsible for executing a multi-asset vault strategy: a vault curator, portfolio manager, protocol strategist, or treasury/asset-management operator. The end depositor is not the initial Setpoint operator.

5.2 Initial ICP

ICP	Why it fits	Need intensity	MVP priority
Tokenized-stock / RWA index vaults	Multi-asset targets, market-hour asymmetry, oracle/risk constraints, RWA ecosystem fit.	High	P0
Public multi-asset ERC-4626 style vaults	Continuous NAV/share priceability and recurring deposits/withdrawals make orchestration visible, provided the vault already has a clean authoritative accounting model.	High	P0/P1
Managed partner vaults	Larger trade sizes and execution constraints, but potentially lower frequency and different accounting.	Medium	Research
Stablecoin allocation vaults	More natural opposing flow and lower valuation complexity; useful later for cross-vault optimization.	Medium	P2
Retail self-balancing portfolios	Clear “pie portfolio” UX, but not the first B2B infrastructure wedge.	Medium	Future

5.3 Jobs to be done

- When my vault drifts from its policy, tell me whether I should rebalance now and what the next safe trade batch is.
- Use current executable liquidity rather than a static NAV cap to size each leg.
- Keep the vault inside risk, oracle, slippage, and valuation constraints after every completed step.
- Re-evaluate after fills, deposits, withdrawals, and price changes instead of trusting an outdated initial plan.
- Use new deposits to reduce drift before creating unnecessary sells when policy permits.
- Let my existing execution infrastructure move funds; do not force me to trust a new custody system.
- Give me an auditable reason why the solver chose each leg and why alternatives were rejected.

6. Validation evidence and research synthesis

The following evidence was collected before implementation. It is directional product research, not a claim that every participant endorses Setpoint. Private-message wording should be reviewed before public attribution.

Source	Signal	Product implication
Carson Brown / Gauntlet perspective	Vaults rebalance at different cadences; response time may be the edge; opposing flow is not reliable. Vault positions must remain priceable across potential states.	Do not depend on cross-vault netting. Use explicit priceable checkpoints and stepwise re-solving.
RWA Index founder / Robinhood Chain	Venue-vs-oracle divergence was testnet-specific, but static 10% NAV sizing was too tight on deep pools and too loose on thin pools. Asked for current weights + targets + constraints + liquidity → biggest safe batch.	Depth-aware sizing is the first concrete wedge and benchmark target.
Kelthar / CoW contributor feedback	Final-state-to-swaps alone is not especially novel. Automatic self-balancing policy feels different. Use trusted CoW primitives for fund handling; hooks/TWAP/limit orders can be components.	Build policy/solver glue above execution. Treat target manipulation as a core security threat.
Ox Developer Support	Public Swap API is swap-level, not portfolio-level. State can change between legs; re-evaluate residual delta and refresh quotes. Portfolio logic sits above Ox.	Setpoint owns orchestration; Ox can be an execution adapter.
Gus / Concrete	Rebalancing depends on vault architecture. Do not make Setpoint responsible for multi-asset accounting: use a single accounting/deposit asset where practical, convert external deposit assets upstream, or separate different exposures into independently accounted sleeves/sub-vaults.	Freeze accounting as an upstream integration prerequisite. Setpoint reads authoritative NAV/positions and owns only policy, solver, and rebalance orchestration.

6.1 Evidence quality

The strongest evidence is the RWA Index integration target because it includes an actual contract interface, guard conditions, addresses, and fork instructions. The rest is expert directional feedback that shaped architecture and falsified assumptions. The MVP must now convert this research into measured evidence.

7. Product principles

Single-vault value first

Setpoint must create value for one vault without requiring another vault, solver network, or counterparty to appear.

State before plan

Always solve from the current confirmed portfolio state. Never assume previous quotes or intended fills remain true.

Explicit priceability

Every completed step must leave holdings that the integrated vault's own accounting model can value under its declared numeraire. Setpoint consumes that accounting; it does not define or reconcile it.

Policy over prediction

The curator defines allowed outcomes and constraints. Setpoint optimizes inside that envelope rather than deciding investment strategy.

Use trusted execution rails

Do not rebuild swapping or settlement unless there is a proven reason. Integrate established execution primitives.

Explain every decision

A solver recommendation must expose constraint checks, expected drift reduction, cost assumptions, and rejected alternatives.

Safety degrades to no-trade

When prices are stale, assets are paused, policy is invalid, or quotes are unsafe, Setpoint should refuse to act rather than improvise.

Benchmark before polish

Measured improvement over baseline is a release gate. UI quality cannot compensate for a solver that does not improve execution.

8. Scope: MVP, V1, future, and explicit non-goals

8.1 MVP - Buildathon proof

- RWA Index fork harness running locally with refreshed oracle and synchronized test pools.
- Baseline implementation reproducing the vault's current static sizing behavior.
- Setpoint solver v1 that reads current holdings, targets, guards, prices, and a liquidity/depth model.
- Adaptive per-leg sizing that produces an RWA Index-compatible Trade[] output.
- Simulation gate: every recommendation is tested against the vault before it is considered executable.
- Stepwise execution/re-solving: after a successful simulated or executed step, recompute from the new state.
- Benchmark report comparing baseline and Setpoint across multiple controlled scenarios.
- Minimal operator UI showing state, target ranges, constraints, recommendation, expected improvement, and simulation result.
- Policy safety controls in the Setpoint layer: allowlisted assets, weight caps, stale-data checks, and change validation.
- Reproducible test scripts and a README explaining what works, what is mocked, and what remains unsupported.

8.2 V1 after MVP

- One external production-style execution adapter (CoW or 0x) behind a common interface.
- Deposit-aware drift correction before selling existing positions.
- Configurable trigger policy: drift threshold, cost threshold, cooldown, and execution window.
- Historical replay / backtest mode using captured prices and depth snapshots.
- Market-hours-aware policy for tokenized equities.
- Corporate-action detection and asset pause/normalization workflow.
- Policy-change governance hooks for multisig/timelock or vault-specific authorization.

8.3 Future

- Cross-vault netting when enough simultaneous opposing flow exists.
- Solver competition or third-party solver network.
- Advanced optimal-control / multi-period execution objectives.
- Multiple accounting numeraires and cross-currency vaults.
- Institutional reporting, attribution, and SLA features.
- Retail "pie portfolio" product built on top of Setpoint infrastructure.

8.4 Explicit non-goals for MVP

- Building a new DEX or AMM.
- Custodying user funds in Setpoint.
- Building cross-vault internal settlement.
- Supporting every ERC-4626 vault generically on day one.
- AI/LLM as a core execution dependency.
- Guaranteeing exact final weights.
- Replacing the vault's accounting system.
- Claiming production readiness for real user funds without review/audit.

- Designing generic multi-asset NAV/share/reward accounting or reconciliation.
- Handling arbitrary user deposit/withdraw conversion inside Setpoint; upstream swap/deposit tooling or the vault owns that flow.
- Choosing whether a strategy should use one vault or multiple independently accounted sleeves/sub-vaults.

9. Product experience and user flows

9.1 Primary operator flow

1. Connect / select vault
2. Read current holdings, NAV, targets, policy, oracle status
3. Set or confirm target ranges and risk limits
4. Setpoint detects drift / receives manual rebalance request
5. Fetch or model executable liquidity
6. Generate candidate trade sets
7. Score candidates against constraints + objective
8. Simulate the chosen batch against vault guards
9. Present recommendation and rationale
10. Execute or export Trade[]
11. Confirm resulting state
12. Recompute drift and next action
13. Stop when target region reached or policy says no further trade

9.2 Drift-triggered automation

Automation is policy-driven, not an uncontrolled agent. A rebalance can be considered only when drift breaches a configured threshold, prices are fresh, the vault is not paused, the asset universe is allowed, estimated benefit exceeds execution cost, and the next step passes simulation.

9.3 Deposit-aware path

When new base-asset deposits arrive, Setpoint should first test whether directing those funds into underweight positions reduces drift enough to avoid selling overweight positions. This is expected to reduce unnecessary turnover and is a high-value V1 optimization.

9.4 Failure flow

- ```
Quote / simulation / execution fails
-> do not reuse stale plan
-> refresh vault state
-> refresh prices and liquidity
-> classify failure
-> recompute candidate batch
-> stop if constraints or data quality remain invalid
```

## 10. Functional requirements

| ID     | Requirement              | Acceptance summary                                                                                                                                                                       |
|--------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-001 | Vault state ingest       | Read assets, balances, target weights/ranges, base asset, authoritative NAV/drift inputs, policy values, and authorization state from the vault/adaptor. Do not invent share accounting. |
| FR-002 | Price ingest             | Read price + timestamp for every relevant asset; reject stale or invalid values.                                                                                                         |
| FR-003 | Liquidity ingest         | Receive a depth/quote model per venue and asset pair; support mocked depth in the first benchmark.                                                                                       |
| FR-004 | Target model             | Support target weights and acceptable ranges rather than requiring exact equality.                                                                                                       |
| FR-005 | Policy model             | Support allowed assets, max weight, max leg size, slippage bounds, max NAV loss, cooldown, and optional market window.                                                                   |
| FR-006 | Candidate generation     | Generate feasible sell and buy legs using the base asset as the hub in the first integration.                                                                                            |
| FR-007 | Adaptive sizing          | Size each leg based on drift, depth, quote impact, and vault constraints rather than one global static percentage.                                                                       |
| FR-008 | Objective scoring        | Rank candidate batches by feasibility first, then drift reduction and execution efficiency.                                                                                              |
| FR-009 | Simulation               | Simulate the proposed batch against the target vault before execution/export.                                                                                                            |
| FR-010 | Explainability           | Return expected drift before/after, expected cost, active constraints, and why each leg size was selected.                                                                               |
| FR-011 | Stepwise re-solve        | After every confirmed step, discard old assumptions and solve again from the new state.                                                                                                  |
| FR-012 | Export                   | Output a vault-compatible Trade[] for RWA Index.                                                                                                                                         |
| FR-013 | Execution adapter        | Expose a common adapter interface for future CoW/Ox/native routes.                                                                                                                       |
| FR-014 | Policy-change validation | Reject unallowlisted assets, excess target weights, or invalid target sums before solver execution.                                                                                      |
| FR-015 | No-trade result          | Return a typed no-trade reason when benefit is insufficient or constraints/data prevent safe execution.                                                                                  |
| FR-016 | Benchmark mode           | Run baseline and Setpoint under identical scenario inputs and persist metrics.                                                                                                           |

# 11. Solver specification

## 11.1 Solver inputs

| Input group       | Minimum fields                                                                                  |
|-------------------|-------------------------------------------------------------------------------------------------|
| Portfolio state   | balances, current weights, base-asset balance, NAV, current drift                               |
| Targets           | target weight or range per asset, cash/base target                                              |
| Vault constraints | max leg value, max NAV loss, slippage floor, allowed assets, stale-price limit                  |
| Prices            | reference price, timestamp, source metadata                                                     |
| Liquidity         | venue, pair, executable depth / quote curve, fees, quote expiry                                 |
| Policy            | rebalance threshold, max turnover, optional minimum expected benefit, optional execution window |

## 11.2 Solver output

```
RebalancePlan {
 vault
 stateHash / blockNumber
 trades: Trade[]
 expectedDriftBefore
 expectedDriftAfter
 expectedTurnover
 expectedCost
 activeConstraints[]
 rejectedAlternatives[]
 simulationResult
 reason / noTradeReason
}
```

## 11.3 MVP objective hierarchy

The solver objective is lexicographic in spirit even if implemented with simpler heuristics in v1. A recommendation that violates a hard vault constraint is never “better” because it reduces drift more.

1. Feasibility: all hard policy, vault, price, and simulation constraints must pass.
2. Safety: prefer plans with larger margin to slippage, price, and NAV-loss boundaries when expected improvement is similar.
3. Convergence: maximize reduction in portfolio drift / distance to target region per step.
4. Execution efficiency: minimize expected price impact, fees, and unnecessary turnover for comparable convergence.
5. Operational simplicity: prefer fewer legs and venues when the economic outcome is materially equivalent.

## 11.4 Baseline algorithm

The baseline reproduces a static per-leg sizing policy such as “up to 10% of NAV per leg,” sells overweights into the base asset first, then buys underweights. The baseline is not intentionally weakened; it must match the reference vault behavior as faithfully as practical.

## 11.5 Adaptive sizing v1

Setpoint v1 estimates a maximum safe executable value for each desired leg from the minimum of: amount required to enter the target range, vault per-leg cap, available sell balance, depth/price-impact limit, oracle-

derived minimum-out safety, policy turnover cap, and any venue-specific quote maximum. It then builds a coherent batch, simulates it, and backs off sizes if the target vault rejects the batch.

## 11.6 Exact weights versus ranges

Exact final weights are not a hard requirement because realized execution surplus, fees, and price movement change the state. A target range is the native abstraction: once an asset is inside its acceptable band, additional trading is not required unless another portfolio constraint justifies it.

## 11.7 Candidate backoff

```
candidate batch
-> simulate
 PASS -> eligible plan
 FAIL due to stale price -> stop / refresh
 FAIL due to size/slippage -> reduce affected leg(s) and rebuild
 FAIL due to drift not improved -> rebuild leg ordering/sizes
 FAIL due to NAV loss -> reduce impact / change venue / stop
```

## 12. Vault model, accounting, and priceability

### 12.1 Upstream accounting prerequisite

Setpoint is not a vault accounting protocol. Before a vault is considered compatible, its own contracts or adapter must expose an authoritative valuation model: base numeraire, holdings, NAV or equivalent portfolio value, and any share/deposit/redemption rules that affect execution. If those values are ambiguous, Setpoint returns unsupported or no-trade rather than constructing a parallel accounting system.

### 12.2 MVP pattern: one numeraire, multiple strategy positions

The first integration follows a simple pattern: one base accounting asset, initially USDC-like, while the strategy may hold multiple underlying positions. NAV, solver costs, cash buffers, and policy limits are expressed in that numeraire. This matches the RWA Index test case and keeps Setpoint focused on execution intelligence rather than user-level asset accounting.

### 12.3 Alternative architecture: independent sleeves/sub-vaults

For economically different exposures such as WBTC and WETH, an integrating product may choose independently accounted sleeves/sub-vaults instead of forcing one monolithic accounting model. Setpoint can treat each sleeve's authoritative value or share token as an input to higher-level allocation policy, but it does not calculate the sleeve's rewards, liabilities, or internal share price. This pattern is compatible with the architecture but is not required for the MVP benchmark.

### 12.4 Deposit and withdrawal conversion is upstream

If users arrive with assets other than the strategy/base asset, swap-and-deposit infrastructure such as Enso can normalize those deposits before they enter the vault. On withdrawal, the vault or upstream product can convert back according to its own user promise. Setpoint does not own that conversion path. It only respects cash/redemption buffers exposed by the vault when sizing a rebalance.

## 12.5 Continuous priceability

A public vault must be able to compute its value from actual confirmed holdings after every completed rebalance step. Setpoint therefore avoids an opaque transition state whose value depends on future promised fills. If a full rebalance cannot be completed safely in one step, Setpoint commits or exports a smaller completed transition and then re-solves from the new authoritative state.

## 12.6 Redemption liquidity

Redemption and payout logic remain the vault's responsibility. Setpoint may consume a configured minimum cash buffer or reserve requirement, but it must not size a rebalance that violates the vault's declared payout policy. No Setpoint component should silently redefine what depositors are owed.

# 13. Execution architecture and adapters

## 13.1 Architectural boundary

```

Vault Accounting + Curator Policy
 |
 v
Setpoint Solver
(state + targets + constraints + liquidity)
 |
 v
Rebalance Plan / Trade[]
 |
 +--> Native vault adapter (MVP test)
 +--> CoW adapter (future/optional)
 +--> 0x adapter (future/optional)
 |
 v
Execution / Settlement
 |
 v
Confirmed new vault state
 |
 +-----> Setpoint re-solves

```

## 13.2 Native RWA Index adapter

The first adapter maps Setpoint output directly into the external vault's `rebalance(Trade[])` interface and uses simulation as a hard gate. This is sufficient to prove solver value before integrating a separate execution venue API.

## 13.3 0x adapter considerations

- 0x remains swap-level; Setpoint computes portfolio residuals and next legs.
- Refresh quotes after portfolio state changes.
- Support exact-buy where a precise acquisition amount is useful.
- Consider sellEntireBalance for execution-time balance uncertainty where compatible.
- Do not assume multiple 0x swaps are collectively atomic.
- Robinhood RWA access and permissions must be handled separately from solver logic.

## 13.4 CoW adapter considerations

- Use CoW settlement primitives for vulnerable fund-moving operations where feasible.

- Limit orders, TWAPs, smart orders, and hooks may be components, not substitutes for Setpoint policy logic.
- A post-swap hook or external observer can trigger state re-check/re-solving.
- Large-volume trades may choose time-sliced execution rather than one immediate swap when policy allows.

## 14. Security and threat model

Setpoint changes economic actions, so security must cover more than direct theft. Policy manipulation, stale prices, malicious routing, and unsafe target changes can destroy value without a classic “funds transferred to hacker” exploit.

| Threat                            | Failure mode                                                                       | Mitigation                                                                                                                              | Priority |
|-----------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------|
| Malicious target update           | Attacker changes target to 100% attacker-controlled asset.                         | Asset allowlist; max per-asset weight; governance authorization; target-change bounds; optional timelock; policy diff preview.          | P0       |
| Stale oracle                      | Solver sizes trades on old prices.                                                 | Timestamp checks; hard stale cutoff; no-trade on invalid data; source metadata.                                                         | P0       |
| Oracle/venue divergence           | Execution price and reference diverge beyond policy.                               | Simulation; minOut; impact cap; venue freshness; optional market-hours policy.                                                          | P0       |
| Thin liquidity / price impact     | Nominally valid leg fails or causes excess loss.                                   | Depth-aware sizing; quote curves; backoff; max NAV loss; simulation.                                                                    | P0       |
| State race                        | Vault changes after solve but before execution.                                    | Bind plan to block/state hash when possible; re-read before execution; short validity window.                                           | P0       |
| Partial multi-step execution      | Earlier leg changes later requirements.                                            | Treat each completed step as final state; discard remaining stale plan; re-solve.                                                       | P0       |
| Unauthorized execution            | Unknown actor calls rebalance.                                                     | Respect vault authorization/session model; Setpoint does not bypass it.                                                                 | P0       |
| Bad adapter                       | Execution adapter redirects or misprices trades.                                   | Adapter allowlist; reviewed contracts; simulation; constrained approvals; minimal trust surface.                                        | P1       |
| Corporate action mismatch         | Oracle and token/pool normalization update at different times.                     | Detect split/event; pause asset; normalize only after verified state alignment.                                                         | P1       |
| Market closed reference ambiguity | Underlying is closed while onchain token trades on news.                           | Market-hours policy; price-source confidence; tighter/no-trade windows if reference is stale by design.                                 | P1       |
| DoS / pathological asset list     | Too many assets or routes make solving/simulation expensive.                       | Bound asset count, venue count, and candidate set in MVP.                                                                               | P1       |
| Accounting model mismatch         | Solver assumes balances/NAV/share semantics that differ from the integrated vault. | Adapter reads authoritative vault accounting; compatibility checks; unsupported/no-trade on ambiguity; no parallel Setpoint accounting. | P0       |

### 14.1 Approval/custody principle

Setpoint should not maintain broad custody or unlimited token approvals. Where adapters require approvals, permissions should be narrowly scoped and delegated to established execution contracts or the vault itself.

## 15. Technical architecture

### 15.1 Components

| Component         | Responsibility                                                                                                 | MVP form                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Vault Reader      | Read authoritative balances, base asset, NAV/value inputs, targets, constraints, prices, drift, authorization. | RWA Index-specific reader; accounting remains vault-owned |
| Policy Engine     | Validate allowed assets, ranges, limits, trigger conditions.                                                   | TypeScript/Rust service + shared types                    |
| Liquidity Model   | Represent executable depth, quotes, fees, expiry.                                                              | Deterministic test model / pool reserves                  |
| Solver            | Generate and rank feasible next-step plans.                                                                    | Deterministic offchain solver                             |
| Simulator         | Call vault with proposed Trade[] and capture revert/pass.                                                      | Anvil fork + eth_call / local RPC                         |
| Execution Adapter | Submit/export plan to venue/vault.                                                                             | RWA Index native interface                                |
| State Monitor     | Observe confirmed state and trigger re-solve.                                                                  | CLI/service polling in MVP                                |
| Operator UI       | Display portfolio, policy, plan, explanation, benchmark.                                                       | Minimal web app                                           |
| Benchmark Runner  | Run scenario matrix and output metrics.                                                                        | Script + JSON/CSV result artifacts                        |

### 15.2 Onchain versus offchain

The MVP solver can be offchain because the key proof is portfolio-level decision quality and safe integration. Critical fund movement and vault invariants remain enforced by the target vault or trusted execution contracts. Onchain verification of a richer optimization certificate can be explored later if it provides real security or trust-minimization value; it is not required to prove the wedge.

### 15.3 Determinism

Given the same state, prices, liquidity model, and policy, the solver should produce deterministic output. This is important for reproducible benchmarks, debugging, and judge-facing explanation.

## 16. Data model and interfaces

### 16.1 Core types

```
AssetPolicy { token, targetMin, targetMax, maxWeight, enabled }
VaultPolicy { baseAsset, driftTrigger, maxTurnover, maxNavLoss, maxPriceAge, allowedVenues }
PortfolioState { blockNumber, nav, balances[], weights[], drift, cashWeight }
PricePoint { token, priceWad, updatedAt, source }
LiquidityQuote { venue, tokenIn, tokenOut, amountIn, expectedOut, minOut, fee, expiry }
Trade { tokenIn, tokenOut, amountIn, minAmountOut }
RebalancePlan { trades[], expectedDriftAfter, expectedCost, simulation, reason }
```

### 16.2 Adapter interface

```
interface ExecutionAdapter {
 quote(intent): Quote[]
 simulate(plan): SimulationResult
 execute?(plan): TxResult
 normalizeReceipt?(tx): ExecutionReceipt
}
```

### 16.3 Solver API

```
POST /solve
{ state, targets, constraints, prices, liquidity }

-> { plan | noTrade, metrics, explanation }
```

## 17. RWA Index integration plan

### 17.1 External interface

The RWA Index vault exposes a `rebalance(Trade[] calldata trades)` function. Each trade contains `tokenIn`, `tokenOut`, `amountIn`, and `minAmountOut`. The vault enforces hard constraints and reverts the full batch if any requirement fails.

Accounting fit: the reference vault already uses a USDC-like base asset and values its basket through its own oracle/NAV logic. Setpoint consumes those outputs; the benchmark does not replace or reimplement the vault's share accounting.

### 17.2 Integration steps

6. Clone/fork the RWA Index repository and build the contracts/agent locally.
7. Create an Anvil fork of Robinhood Chain testnet or reproduce the deployment state locally.
8. Impersonate the provided admin/manager for simulation-only calls.
9. Refresh stale oracle prices using the authorized feeder role in the fork.
10. Synchronize toy pools to the oracle using the provided syncer logic so the baseline is not blocked by intentionally stale testnet state.
11. Read `assets()`, `targetWeight(token)`, `cashTarget()`, current balances, and policy guard parameters.
12. Run the current/static baseline through the same simulation path.
13. Generate Setpoint `Trade[]` output and simulate it using the same vault state and liquidity snapshot.
14. Persist pass/fail, drift, turnover, number of steps, and failure reasons.
15. Repeat across a scenario matrix rather than presenting one cherry-picked case.

### 17.3 What this integration proves

- Setpoint can target a contract it did not design.
- Setpoint output respects external hard invariants.
- The solver can be evaluated against a real user-provided interface.
- The benchmark is reproducible and falsifiable.

## 18. Benchmark and evaluation framework

### 18.1 Primary benchmark question

#### Benchmark

Compared with a truthful static sizing baseline, does Setpoint reach the target region with fewer failed simulations, fewer steps, less time off-target, and equal or lower economic cost under the same constraints?

### 18.2 Metrics

| Metric                   | Definition                                                                                 | Direction                        |
|--------------------------|--------------------------------------------------------------------------------------------|----------------------------------|
| Steps to target region   | Number of successful rebalance batches until all required weights are inside target bands. | Lower                            |
| Simulation failure rate  | Failed candidate simulations / attempted candidates.                                       | Lower                            |
| Drift reduction per step | $(\text{drift\_before} - \text{drift\_after}) / \text{drift\_before}$ .                    | Higher                           |
| Time / cycles off-target | Number of policy cycles the portfolio remains outside target region.                       | Lower                            |
| Turnover                 | Gross traded notional / NAV.                                                               | Lower for equivalent convergence |
| Estimated execution cost | Fees + price impact + route costs.                                                         | Lower                            |
| Safety margin            | Distance from slippage/NAV-loss hard limits.                                               | Higher within reason             |
| Recompute stability      | Whether the next solve remains valid after realistic state changes.                        | Higher                           |

### 18.3 Scenario matrix

- Deep liquidity + moderate drift.
- Thin liquidity + moderate drift.
- Asymmetric liquidity: one deep asset, one thin asset.
- Large target change that cannot be completed safely in one step.
- Cash-target increase / defensive preset.
- Fresh deposit that can be used to correct underweights.
- One venue quote expires / changes between steps.
- Stale oracle → expected no-trade.
- Corporate-action mismatch → expected pause/no-trade in future module.
- Market-closed tokenized-stock reference ambiguity → policy response test in future module.

### 18.4 Evidence standard

No benchmark number is to be written into the pitch or README until it is generated by a reproducible script and can be re-run. Negative results are useful: if adaptive sizing does not outperform the baseline in a scenario, the PRD should record why instead of hiding it.

## 19. Testing and quality requirements

### 19.1 Unit tests

- Weight/range calculations
- Drift calculation and target-region detection
- Per-leg sizing caps
- Liquidity-depth interpolation / quote handling
- Policy validation
- No-trade reasons
- Candidate backoff logic
- Deterministic plan generation

### 19.2 Integration tests

- RWA Index simulation pass
- TradeTooLarge-style rejection handling
- SlippageTooLoose-style rejection handling
- DriftNotImproved handling
- ExcessiveValueLoss handling
- StalePrice handling
- Unauthorized caller/session handling
- State changes between solve and simulate

### 19.3 Invariants

- Solver never outputs an unallowlisted asset.
- Target ranges must be internally consistent.
- A successful Setpoint step must not intentionally increase drift unless a higher-priority safety/cash constraint explicitly requires it.
- No plan is marked executable without simulation or a documented reason simulation is unavailable.
- Re-solving uses confirmed state, not expected state.
- All monetary comparisons use the integrated vault's declared authoritative numeraire in MVP.
- Setpoint never invents depositor/share accounting; it consumes the integrated vault's authoritative accounting interface.
- If base numeraire, NAV semantics, or payout rules cannot be represented unambiguously by an adapter, the integration is unsupported.

### 19.4 README honesty requirements

- Clearly label fork-only behavior.
- Clearly label toy pool depth.
- Do not imply real Robinhood Stock Token liquidity if using seeded/mock pools.
- List deployed addresses only when actually deployed.
- List known limitations and unsupported paths.
- Include exact benchmark command and expected artifact locations.

## 20. Observability, analytics, and success metrics

### 20.1 Operator telemetry

- Vault state block / timestamp
- Oracle freshness
- Current drift
- Trigger status
- Candidate count
- Chosen plan and rejected alternatives
- Simulation result and revert classification
- Expected vs realized outputs
- Drift after execution
- Cumulative turnover and estimated execution cost

### 20.2 Product-level success metrics

| Metric               | MVP success signal                                                                                                |
|----------------------|-------------------------------------------------------------------------------------------------------------------|
| External integration | At least one real third-party vault interface integrated and reproducibly simulated.                              |
| Measured improvement | Setpoint beats static sizing on at least one meaningful liquidity scenario without worsening hard safety metrics. |
| Robustness           | Handles at least four deliberate failure modes cleanly and explainably.                                           |
| Validation           | External builder can inspect a tx/simulation trace or benchmark artifact without running a custom app.            |
| Reproducibility      | Judge/reviewer can run baseline and Setpoint comparison from README.                                              |
| Narrative            | Clear before/after story: weak netting assumption killed by research → concrete solver wedge → measured proof.    |

## 21. GTM, design partners, and business model hypotheses

### 21.1 Beachhead GTM

The first GTM motion is not broad retail acquisition. It is direct integration with vault teams that already have recurring rebalances, explicit risk rules, and an authoritative accounting boundary. The product should enter as a solver/automation component that can be benchmarked against the team's current process before asking them to migrate execution. Vaults whose NAV/share semantics are still undefined are not initial Setpoint customers.

### 21.2 Design-partner loop

1. Ask for last real rebalance workflow
2. Reproduce baseline
3. Run Setpoint on the same constraints
4. Send result / simulation / tx trace
5. Ask: would you trust this recommendation and what blocks integration?
6. Fix the blocker
7. Repeat

### 21.3 Switching reason

Setpoint should not sell “a few bps better” as its only benefit. The switching case is operational: fewer unnecessary cycles, safer sizing against real liquidity, automatic re-evaluation after state changes, and a policy layer that keeps vault execution auditable and priceable.

### 21.4 Business model hypotheses

- B2B SaaS / API fee for solver and automation.
- Usage fee based on managed NAV or successful rebalance volume, only if aligned with users.
- Enterprise integration and monitoring tier for managed vaults.
- Self-hosted or licensed solver for protocols that require their own infrastructure.
- No token is required for the initial business model.

## 22. Competitive positioning

| Category / product                      | What it does well                                                                           | Setpoint boundary / differentiation                                                                                                                            |
|-----------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x Swap API                             | Individual swap routing across liquidity, RFQ, multihop.                                    | Setpoint decides portfolio-level next actions and re-solves after state changes; 0x can execute legs.                                                          |
| CoW Protocol                            | Batch settlement, intents, solvers, smart orders, TWAP/limit primitives, trusted execution. | Setpoint supplies vault policy, target ranges, adaptive sizing, trigger logic, and portfolio-level orchestration.                                              |
| Enso-like swap/deposit routing          | Convenient asset conversion and deposit composition.                                        | Useful upstream deposit/withdraw conversion and composition; Setpoint starts after the vault/accounting boundary and remains portfolio rebalance intelligence. |
| Vault-native agent scripts              | Strategy-specific automation tightly coupled to one vault.                                  | Setpoint aims to become a reusable solver/orchestration layer with adapter boundaries and benchmarks.                                                          |
| Cross-vault netting / clearing concepts | Potential turnover reduction if opposing flow exists.                                       | Future optional optimization; not required for Setpoint to create value.                                                                                       |
| Traditional "pie portfolio" rebalancing | Simple target-weight UX and periodic rebalance.                                             | Setpoint adapts execution to onchain liquidity, policy constraints, priceability, and programmable venues.                                                     |

### 22.1 Positioning statement

#### Positioning

Setpoint sits between portfolio policy and swap execution. It does not tell a manager what to own; it decides how to safely move the vault toward the policy they already chose.

## 23. Build plan and milestone schedule

The deadline context is short. The build must prioritize falsifiable technical proof over breadth.

| Milestone | Focus                     | Exit criterion                                                             | Target            |
|-----------|---------------------------|----------------------------------------------------------------------------|-------------------|
| M0        | PRD freeze + repo hygiene | PRD v1.1, README skeleton, architecture, issue list                        | Day 0             |
| M1        | RWA Index harness         | Fork boots; oracle refresh; pool sync; rebalance simulation works          | Day 1             |
| M2        | Baseline                  | Static sizing scenarios produce reproducible metrics                       | Day 1-2           |
| M3        | Solver v1                 | Produces feasible Trade[] and typed no-trade outcomes                      | Day 2-3           |
| M4        | Adaptive sizing           | Depth-aware sizing + backoff passes target-vault simulations               | Day 3-4           |
| M5        | Benchmark                 | Scenario matrix; charts/tables; failure modes; reproducible artifacts      | Day 4-5           |
| M6        | Operator UI               | State → policy → recommendation → simulation → next state                  | Day 5-6           |
| M7        | Security + docs           | Threat model, deliberate failures, README truthfulness, tests              | Day 6-7           |
| M8        | Submission                | Demo video, pitch, deployment/integration evidence, Founder House material | Before submission |

### 23.1 Priority rule under time pressure

If schedule slips, cut breadth in this order: second execution adapter → market-hours module → corporate-action module → advanced UI. Never cut the external-vault benchmark, solver, simulation gate, or failure-mode proof.

## 24. Demo, judging, and Founder House narrative

### 24.1 Demo story

16. Show the external RWA Index vault and its real guard conditions.
17. Show a portfolio target change and the static baseline sizing.
18. Run a scenario where static sizing is inefficient or fails against liquidity/guard constraints.
19. Run Setpoint on exactly the same state.
20. Show the recommended Trade[], why each size was chosen, and the simulation pass.
21. Execute/simulate and show the resulting explicit priceable state.
22. Re-solve from that new state and show convergence.
23. Trigger one deliberate failure, such as stale price or malicious target, and show Setpoint refusing to act.
24. Close with benchmark results and the external design-partner path.

### 24.2 Three-sentence pitch

#### Pitch

Vault managers already know their target allocation; the hard part is executing toward it safely as liquidity, prices, and portfolio state change. Setpoint turns target ranges and risk constraints into the best next executable rebalance step, re-solving after every confirmed state instead of relying on static leg sizes or stale multi-leg plans. We are proving it against an external Robinhood Chain RWA index vault, benchmarking adaptive sizing directly against its existing static approach.

### 24.3 Why Arbitrum / Robinhood Chain

- Initial design partner/test case is already a Robinhood Chain RWA index vault.
- Tokenized-stock vaults expose execution and valuation problems that fit the product wedge.
- Arbitrum ecosystem already has mature execution primitives Setpoint can compose instead of rebuilding settlement.
- Founder House thesis is product-to-startup progression: Setpoint has research, a real integration surface, measurable proof, and a direct path to more vault partners.

### 24.4 What not to say

- Do not claim Setpoint “solves MEV” broadly.
- Do not claim production institutional readiness before audit/real-liquidity tests.
- Do not present seeded testnet pool results as real market liquidity.
- Do not claim cross-vault netting as validated demand.
- Do not call CoW/Ox competitors when they are likely execution infrastructure partners.
- Do not hide failed benchmark scenarios.

## 25. Risks, kill criteria, and open decisions

### 25.1 Product risks

| Risk                       | Why it matters                                                                                                               | Response                                                                                                                                                                   |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sizing benefit too small   | If adaptive sizing barely changes outcomes, the wedge is weak.                                                               | Run benchmark early; pivot objective or ICP only if evidence demands it.                                                                                                   |
| Liquidity data quality     | Solver quality is bounded by quotes/depth inputs.                                                                            | Make data source explicit; keep no-trade path; support multiple adapters later.                                                                                            |
| Vault heterogeneity        | Every vault exposes different constraints/interfaces.                                                                        | Start adapter-first with one real vault; define common abstraction only after two integrations.                                                                            |
| Accounting complexity      | Multi-asset products can hide complex NAV, reward, redemption, and reconciliation semantics that are not execution problems. | Treat accounting as upstream. Require authoritative adapter inputs; prefer single numeraire for MVP; allow independently accounted sleeves; reject ambiguous integrations. |
| Execution venue dependency | CoW/0x support and permissions vary by chain/asset.                                                                          | Keep execution adapter modular; native test adapter first.                                                                                                                 |
| RWA compliance/access      | Real RWA assets may require opt-in/legal eligibility.                                                                        | Keep testnet benchmark independent; treat production access as GTM/integration requirement.                                                                                |
| Overbuilding               | Too many modules dilute proof.                                                                                               | Protect benchmark + external integration as release gates.                                                                                                                 |

### 25.2 Kill criteria

- Across credible scenarios, Setpoint cannot outperform or materially de-risk a simple static policy.
- Real vault managers consistently state that current execution is not painful and would not integrate even with measurable improvement.
- The external integration requires Setpoint to take custody or bypass core vault safety controls.
- A mature existing product already provides the same reusable policy + adaptive solver + stepwise re-evaluation layer and is clearly sufficient for the ICP.

### 25.3 Open decisions - must be resolved by evidence, not preference

| Decision                    | Current default                      | Evidence needed                                                             |
|-----------------------------|--------------------------------------|-----------------------------------------------------------------------------|
| First ICP subtype           | Public multi-asset/RWA vault         | Responses from Rockaway/other vault managers + second integration candidate |
| First external adapter      | Native RWA Index first; CoW/0x next  | Fastest path to real execution evidence and chain/asset availability        |
| Objective weights           | Feasibility → drift reduction → cost | Benchmark sensitivity analysis                                              |
| Onchain solver verification | Not MVP                              | Only add if it materially improves trust or execution safety                |
| Market-hours module timing  | Post-core benchmark                  | Tokenized-stock test showing reference ambiguity is a recurring blocker     |

## 26. Definition of done

### 26.1 Buildathon MVP is done when

- The Setpoint repo is public and non-empty with reproducible setup instructions.
- RWA Index fork/harness is reproducible from a clean environment.
- Baseline and Setpoint are executed against the same scenario inputs.
- Setpoint generates external-vault-compatible Trade[] output.
- At least one meaningful scenario shows measured improvement or materially safer behavior over baseline.
- At least four deliberate failure cases are handled without unsafe execution.
- Every benchmark number in the pitch can be reproduced by a script.
- The UI/demo reflects real solver output, not hardcoded success states.
- Known limitations are clearly documented.
- The RWA Index founder can be sent a result/trace that is useful without needing to test an entire app.

### 26.2 Product is not “done” merely because

- the dashboard looks polished
- the solver returns a trade list
- a single cherry-picked test passes
- the contract deploys
- the idea receives positive verbal feedback

## Appendix A. RWA Index technical facts

These facts are from the external founder-provided integration details and the public repository inspected during product research. They describe a testnet integration target, not Setpoint-owned infrastructure.

| Field                        | Value                                               |
|------------------------------|-----------------------------------------------------|
| Repository                   | github.com/RaYYeR220/rwa-index                      |
| Chain                        | Robinhood Chain testnet                             |
| Chain ID                     | 46630                                               |
| RPC                          | https://rpc.testnet.chain.robinhood.com             |
| Vault                        | 0x357CD10343829DBd5889c7b0B2fBc4388fC4875B          |
| Oracle                       | 0x2F2316d6D4a8952730a71BfCD9fa0Af5A0138AB9          |
| Admin/manager for simulation | 0xbB91Fe38652991f0E9735dc139601bD637ae4d66          |
| Pool syncer                  | 0x1f954000e202EBbB82Da88D53000b3eF0Cd34aeA          |
| Basket                       | TSLA, AMZN, PLTR, NFLX, AMD; USDC hub               |
| Decimals                     | 18 decimals for all test tokens including mock USDC |
| Price units                  | WAD in USDC                                         |

### A.1 Trade interface

```
struct Trade {
 address tokenIn;
 address tokenOut;
 uint256 amountIn;
 uint256 minAmountOut;
}

function rebalance(Trade[] calldata trades) external;
```

### A.2 Onchain guards

- Each leg  $\leq 10\%$  of NAV by oracle value.
- $\text{minAmountOut} \geq \text{oracle-implied output} \times 0.98$ .
- $\text{totalDrift}()$  must strictly decrease.
- NAV after rebalance  $\geq$  NAV before  $\times 0.98$ .
- Oracle prices must be  $\leq 1$  day old.
- Whole batch reverts if any guard or trade fails.

### A.3 Drift definition

$$\text{drift} = ( |\text{cashWeight} - \text{cashTarget}| + \sum |\text{assetWeight}_i - \text{target}_i| ) / 2$$

### A.4 Current testnet gotchas

- Oracle had not been pushed since June at the time of handoff; fork tests should impersonate the authorized feeder and set fresh prices.
- Pools had also not been synchronized; use the provided sync logic / syncer on the fork.
- Pool depth is toy-sized (~4 stock tokens per pool), so it is useful for constraint tests but not representative of real liquidity.
- The vault is permissioned for rebalance; simulation can use the provided manager address.

## Appendix B. Validation log

| Research contact / source | Question tested                                                                                  | What changed in Setpoint                                                                                                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gauntlet / Carson Brown   | Does cross-vault netting work as the wedge?<br>What does a real vault require during transition? | Killed dependency on simultaneous opposing flow; introduced continuous priceability and stepwise execution.                                                                                   |
| RWA Index founder         | What is the real pain in a tokenized-stock index rebalance?                                      | Moved wedge to depth-aware sizing/routing; supplied first external integration and benchmark target.                                                                                          |
| CoW / Kelthar             | Is final-state rebalancing meaningfully different? What should be custom vs primitive?           | Shifted to automatic self-balancing policy; added target ranges, trusted execution rails, target-manipulation threat.                                                                         |
| 0x Developer Support      | Where does portfolio orchestration end and swap routing begin?                                   | Confirmed Setpoint-above-0x boundary and need to refresh state/quotes between legs.                                                                                                           |
| Concrete / Gus            | Is rebalancing itself the problem? How does vault architecture/accounting change it?             | Clarified that Setpoint should not solve vault accounting. Froze authoritative upstream accounting, single-numeraire MVP, optional independent sleeves, and upstream swap/deposit conversion. |
| Rockaway / pending        | Is this recurring enough in RWA/DeFi and what triggers switching?                                | Pending; used to refine ICP and GTM, not to pause core build.                                                                                                                                 |

## Appendix C. Decision log

| Decision                      | Rejected alternative                          | Reason                                                                                                                                                                                                 |
|-------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Single-vault value first      | Cross-vault netting as core                   | Cadence and direction assumptions are unreliable; waiting can destroy edge.                                                                                                                            |
| Adaptive sizing               | Static NAV-per-leg sizing                     | Real test case showed static cap can be too conservative or too aggressive.                                                                                                                            |
| Target ranges                 | Exact final state                             | Execution surplus, fees, and moving prices make exact equality brittle.                                                                                                                                |
| Stepwise re-solve             | One opaque multi-step plan                    | Quotes/state change; vault must remain priceable.                                                                                                                                                      |
| External execution primitives | Custom settlement stack                       | Fund-moving contracts are the highest-risk surface; existing rails already solve execution well.                                                                                                       |
| Accounting upstream           | Setpoint-owned generic multi-asset accounting | Vault/accounting architecture is strategy-specific. Setpoint should consume authoritative NAV/value inputs; MVP uses one numeraire, while different exposures may use independently accounted sleeves. |
| Corporate actions as module   | Corporate actions as core wedge               | Important but infrequent; safety feature, not daily recurring pain.                                                                                                                                    |
| Market-hours aware as module  | Market hours as core MVP                      | Real RWA concern, but solver sizing is the more recurring first proof.                                                                                                                                 |
| No token                      | Protocol-token-first model                    | No demonstrated need; would distract from product validation.                                                                                                                                          |

## Appendix D. Submission checklist

- ☐ Repo README explains problem, architecture, external integration, exact limitations.
- ☐ External RWA Index repo/contract attribution is clear; no ownership confusion.
- ☐ Baseline command documented.
- ☐ Setpoint benchmark command documented.
- ☐ At least one result artifact committed or linked.
- ☐ All deployed/test addresses labeled with chain/network.
- ☐ Toy liquidity/mock components explicitly labeled.
- ☐ Demo includes one deliberate failure.
- ☐ Security section covers policy manipulation and stale pricing.
- ☐ Pitch uses measured claims only.
- ☐ Founder House application describes research → falsification → external integration → benchmark progression.
- ☐ No claim that CoW/0x endorse Setpoint; describe them as research sources/infrastructure boundaries.
- ☐ No claim of production safety for uncapped real funds without audit/review.
- ☐ RWA Index founder receives a result/trace after first benchmark, respecting their limited time.

**END OF PRD v1.1**